

HDOC Day-40 - Paragraph Operations Using Functions

1. Algorithm

Step 1: Start the program.

Step 2: Declare two paragraph arrays and other required arrays for results.

Step 3: Call readParagraph() twice to read Paragraph 1 and Paragraph 2 using fgets().

Step 4: Display the menu:

1. Find the longest and shortest words in both paragraphs.
2. Remove repeated consecutive spaces from both paragraphs.
3. Convert both paragraphs to title case.
4. Find the frequency of every distinct word in both paragraphs.
5. Exit.

Step 5: Read the user's choice and use a switch statement.

Step 6: For Choice 1, call findLongestAndShortestWords() for each paragraph. Make a temporary copy, split it into words using strtok(), compare word lengths using strlen(), and store the longest and shortest words using strcpy().

Step 7: For Choice 2, call removeRepeatedSpaces() for each paragraph. Scan character by character and copy only one space when consecutive spaces occur.

Step 8: For Choice 3, call convertToTitleCase() for each paragraph. Convert the first alphabetic character of each word to uppercase and the remaining alphabetic characters to lowercase.

Step 9: For Choice 4, call findWordFrequencies() for each paragraph. Convert words to lowercase so comparison is case-insensitive, store distinct words, and count their occurrences.

Step 10: For Choice 5, terminate the program.

Step 11: For any other choice, display an invalid-choice message.

Step 12: Repeat the menu until Exit is selected.

Step 13: Stop.

2. C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000
#define MAX_WORDS 200
#define WORD_LEN 100

void readParagraph(char paragraph[], size_t size);
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[], char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);

void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);
    paragraph[strlen(paragraph)] = '\0';
}

void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[], char shortestWord[])
{
    char temp[MAX], *word;
    strcpy(temp, paragraph);

    word = strtok(temp, " ,.!?:\t");
    if (word == NULL) {
        strcpy(longestWord, "");
        strcpy(shortestWord, "");
        return;
    }
```

```

    }

    strcpy(longestWord, word);
    strcpy(shortestWord, word);

    while (word != NULL) {
        if (strlen(word) > strlen(longestWord))
            strcpy(longestWord, word);
        if (strlen(word) < strlen(shortestWord))
            strcpy(shortestWord, word);
        word = strtok(NULL, " ,.!?:\t");
    }
}

void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[])
{
    int i, j = 0;
    int previousWasSpace = 0;

    for (i = 0; paragraph[i] != '\0'; i++) {
        if (isspace((unsigned char)paragraph[i])) {
            if (!previousWasSpace) {
                modifiedParagraph[j++] = ' ';
                previousWasSpace = 1;
            }
        } else {
            modifiedParagraph[j++] = paragraph[i];
            previousWasSpace = 0;
        }
    }
    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[], char titleCaseParagraph[])
{
    int i;
    int newWord = 1;

    for (i = 0; paragraph[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)paragraph[i];

        if (isalpha(ch)) {
            if (newWord)
                titleCaseParagraph[i] = (char)toupper(ch);
            else
                titleCaseParagraph[i] = (char)tolower(ch);
            newWord = 0;
        } else {
            titleCaseParagraph[i] = paragraph[i];
            if (isspace(ch))
                newWord = 1;
        }
    }
    titleCaseParagraph[i] = '\0';
}

void findWordFrequencies(const char paragraph[])
{
    char temp[MAX], words[MAX_WORDS][WORD_LEN];
    int frequency[MAX_WORDS] = {0};
    int count = 0, i, j, found;
    char *word;

    strcpy(temp, paragraph);
    word = strtok(temp, " ,.!?:\t");

    while (word != NULL) {
        for (i = 0; word[i] != '\0'; i++)
            word[i] = (char)tolower((unsigned char)word[i]);

        found = -1;
        for (j = 0; j < count; j++) {
            if (strcmp(words[j], word) == 0) {
                found = j;
                break;
            }
        }

        if (found == -1 && count < MAX_WORDS) {
            strcpy(words[count], word);
            frequency[count] = 1;
            count++;
        } else if (found != -1) {
            frequency[found]++;
        }

        word = strtok(NULL, " ,.!?:\t");
    }
}

```

```

    for (i = 0; i < count; i++)
        printf("%s : %d\n", words[i], frequency[i]);
}

int main()
{
    char paragraph1[MAX], paragraph2[MAX];
    char longest[WORD_LEN], shortest[WORD_LEN];
    char modified[MAX], titleCase[MAX];
    int choice;

    printf("Enter Paragraph 1: ");
    readParagraph(paragraph1, MAX);
    printf("Enter Paragraph 2: ");
    readParagraph(paragraph2, MAX);

    do {
        printf("\n----- PARAGRAPH OPERATIONS ----- \n");
        printf("1. Find longest and shortest words\n");
        printf("2. Remove repeated spaces\n");
        printf("3. Convert to title case\n");
        printf("4. Find word frequencies\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                findLongestAndShortestWords(paragraph1, longest, shortest);
                printf("\nParagraph 1\nLongest word: %s\nShortest word: %s\n",
                    longest, shortest);

                findLongestAndShortestWords(paragraph2, longest, shortest);
                printf("\nParagraph 2\nLongest word: %s\nShortest word: %s\n",
                    longest, shortest);
                break;

            case 2:
                removeRepeatedSpaces(paragraph1, modified);
                printf("\nModified Paragraph 1: %s\n", modified);
                removeRepeatedSpaces(paragraph2, modified);
                printf("Modified Paragraph 2: %s\n", modified);
                break;

            case 3:
                convertToTitleCase(paragraph1, titleCase);
                printf("\nTitle Case Paragraph 1: %s\n", titleCase);
                convertToTitleCase(paragraph2, titleCase);
                printf("Title Case Paragraph 2: %s\n", titleCase);
                break;

            case 4:
                printf("\nWord frequencies in Paragraph 1:\n");
                findWordFrequencies(paragraph1);
                printf("\nWord frequencies in Paragraph 2:\n");
                findWordFrequencies(paragraph2);
                break;

            case 5:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice. Please enter 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}

```

3. Test Case Table

Test Case	Choice	Input / Condition	Expected Output
TC-1	1	P1: I love programming P2: C is powerful	Longest/shortest words are displayed for both paragraphs.
TC-2	2	P1: Hello world P2: C is easy	Repeated spaces are reduced to single spaces.
TC-3	3	P1: hello WORLD P2: c PROGRAMMING is FUN	Hello World C Programming Is Fun
TC-4	4	P1: C is fun c is easy P2: Hello hello WORLD	Case-insensitive frequency of every distinct word is displayed.
TC-5	5	-	Program exits.
TC-6	9	-	Invalid choice message is displayed.

4. Test Case Evaluation

TC-1: The program tokenizes both paragraphs and compares the length of each word. It correctly stores and displays the longest and shortest words. Test Passed.

```
Enter Paragraph 1: I love programming
Enter Paragraph 2: C is powerful
Enter your choice: 1
```

```
Paragraph 1
Longest word: programming
Shortest word: I
```

```
Paragraph 2
Longest word: powerful
Shortest word: C
```

TC-2: The program scans each paragraph character by character. When several consecutive spaces are found, only one space is copied to the modified paragraph. Test Passed.

```
Enter Paragraph 1: Hello  world
Enter Paragraph 2: C   is   easy
Enter your choice: 2
```

```
Modified Paragraph 1: Hello world
Modified Paragraph 2: C is easy
```

TC-3: The program converts the first alphabetic character of every word to uppercase and the remaining letters to lowercase. Both paragraphs are correctly converted to title case. Test Passed.

```
Enter Paragraph 1: hello WORLD
Enter Paragraph 2: c PROGRAMMING is FUN
Enter your choice: 3
```

```
Title Case Paragraph 1: Hello World
Title Case Paragraph 2: C Programming Is Fun
```

TC-4: Words are converted to lowercase before comparison, so C and c, or Hello and hello, are treated as the same word. The program correctly displays each distinct word with its frequency. Test Passed.

```
Enter Paragraph 1: C is fun c is easy
Enter Paragraph 2: Hello hello WORLD
Enter your choice: 4
```

```
Word frequencies in Paragraph 1:
c : 2
is : 2
fun : 1
easy : 1
```

```
Word frequencies in Paragraph 2:
hello : 2
world : 1
```

TC-5: When choice 5 is selected, the switch executes the Exit case and the loop terminates. Test Passed.

```
Enter your choice: 5
Exiting program...
```

TC-6: Since 9 is outside the available menu choices, the default case displays an invalid-choice message. Test Passed.

Enter your choice: 9

Invalid choice. Please enter 1 to 5.